

20 SQL Interview Questions, Free Sample

SQL

Every question was generated from real code, run for real, and checked against the actual output before being kept. Every wrong answer is explained too, not just the right one.

How this book was built

Most interview question banks are written by hand and quietly carry the mistakes of whoever wrote them. This one was built the other way around. Every code snippet in this book was run for real, and the printed answer key is the printed output, not someone's memory of what it should be. Where two independent ways of running the same snippet disagreed, the question was dropped rather than guessed at.

Each question also explains why the three wrong answers are wrong, not only why the right one is right. That is the part most practice tests skip, and it is the part that actually teaches you something the second time you get a question wrong.

SQL Questions

Block 1, Selecting, Filtering and Sorting

1. What does this query return?

```
CREATE TABLE codes (id INTEGER, a INTEGER, b INTEGER);
INSERT INTO codes VALUES
  (1, 1, 3),
  (2, 2, 3),
  (3, 3, 3),
  (4, 1, 1);

SELECT id FROM codes WHERE (a = 1 OR a = 2) AND NOT b = 3
```

- A. (no rows)
- B. 4
- C. 1
4
- D. 1
2

2. What does this query return?

```
CREATE TABLE catalog (id INTEGER, name TEXT, price INTEGER);
INSERT INTO catalog VALUES
  (1, 'mug', 10),
  (2, 'plate', 50),
  (3, 'bowl', 9),
  (4, 'jug', 51);

SELECT name FROM catalog WHERE price BETWEEN 10 AND 50 ORDER BY name
```

- A. bowl
mug
plate
- B. mug
plate
- C. (no rows)
- D. jug
mug
plate

Block 2, Joins

3. What does this query return?

```

CREATE TABLE machines (id INTEGER, tag TEXT, hours_run INTEGER);
INSERT INTO machines VALUES
  (1, 'M1', 120),
  (2, 'M2', 300),
  (3, 'M3', 480);
CREATE TABLE thresholds (label TEXT, max_hours INTEGER);
INSERT INTO thresholds VALUES
  ('Green', 200),
  ('Yellow', 400),
  ('Red', 600);

SELECT machines.tag, thresholds.label FROM machines JOIN thresholds ON
  machines.hours_run <= thresholds.max_hours ORDER BY machines.tag,
  thresholds.label

```

- A. M1 | Green
M1 | Red
M1 | Yellow
M2 | Red
M2 | Yellow
M3 | Red
- B. M2 | Green
M3 | Green
M3 | Yellow
- C. (no rows)
- D. M1 | Green
M2 | Yellow
M3 | Red

4. What does this query return?

```

CREATE TABLE stations (label TEXT);
INSERT INTO stations VALUES
  ('Gate1'),
  ('Gate2'),
  ('Gate3');
CREATE TABLE shifts (label TEXT);
INSERT INTO shifts VALUES
  ('Morning'),
  ('Evening');

SELECT stations.label, shifts.label FROM stations CROSS JOIN shifts ORDER BY
  stations.label, shifts.label

```

- A. Gate1 | Morning
Gate2 | Evening
- B. (no rows)
- C. Gate1 | Evening
Gate1 | Morning
Gate2 | Evening
Gate2 | Morning
Gate3 | Evening
- D. Gate1 | Evening
Gate1 | Morning

Gate2 | Evening
Gate2 | Morning
Gate3 | Evening
Gate3 | Morning

Block 3, Grouping and Aggregation

5. What does this query return?

```
CREATE TABLE loans (id INTEGER, member TEXT, book TEXT, returned TEXT);
INSERT INTO loans VALUES
  (1, 'sam', 'sql basics', 'yes'),
  (2, 'sam', 'sql advanced', NULL),
  (3, 'sam', 'joins', NULL),
  (4, 'tia', 'sql basics', 'yes');

SELECT member, COUNT(*), COUNT(returned) FROM loans GROUP BY member ORDER BY
member
```

- A. tia | 1 | 1
sam | 3 | 1
- B. sam | 3 | 1
tia | 1 | 1
- C. sam | 2 | 1
tia | 1 | 1
- D. sam | 3 | 3
tia | 1 | 1

6. What does this query return?

```
CREATE TABLE scores (id INTEGER, student TEXT, points INTEGER);
INSERT INTO scores VALUES
  (1, 'ann', 45),
  (2, 'bo', 82),
  (3, 'cy', 91),
  (4, 'dee', 58),
  (5, 'eli', 30);

SELECT CASE WHEN points >= 60 THEN 'pass' ELSE 'fail' END AS result, COUNT(*)
FROM scores GROUP BY result ORDER BY result
```

- A. fail | 3
pass | 2
- B. pass | 2
fail | 3
- C. fail | 2
pass | 3
- D. fail | 3
pass | 3

7. What does this query return?

```
CREATE TABLE ticket_sales (id INTEGER, event TEXT, revenue INTEGER);
INSERT INTO ticket_sales VALUES
  (1, 'jazz', 1200),
  (2, 'jazz', NULL),
  (3, 'rock', NULL),
  (4, 'rock', NULL),
  (5, 'pop', 800);

SELECT event, SUM(revenue) FROM ticket_sales GROUP BY event ORDER BY event
```

- A. pop | 800
jazz | 1200
rock | NULL
- B. jazz | 1200
pop | 800
rock | 0
- C. jazz | 1200
pop | 800
rock | NULL
- D. jazz | NULL
pop | 800
rock | NULL

8. What does this query return?

```
CREATE TABLE withdrawals (id INTEGER, account TEXT, amount INTEGER);
INSERT INTO withdrawals VALUES
  (1, 'A1', 200),
  (2, 'A1', 50),
  (3, 'A1', 20),
  (4, 'A2', 500),
  (5, 'A2', 10);

SELECT account, COUNT(*) FROM withdrawals WHERE amount >= 50 GROUP BY account
HAVING COUNT(*) >= 2 ORDER BY account
```

- A. A1 | 2
A2 | 1
- B. A2 | 1
- C. A1 | 3
A2 | 1
- D. A1 | 2

Block 4, NULL, CASE and Set Operations

9. What does this query return?

```
CREATE TABLE filters (id INTEGER, product TEXT, category TEXT);
INSERT INTO filters VALUES
  (1, 'P1', 'A'),
  (2, 'P2', 'C'),
  (3, 'P3', 'D');

SELECT product FROM filters WHERE category NOT IN ('A', 'B', NULL)
```

- A. P1
P2
P3
- B. P3
- C. P2
P3
- D. (no rows)

10. What does this query return?

```
CREATE TABLE waitlist (id INTEGER, applicant TEXT, priority INTEGER);
INSERT INTO waitlist VALUES
  (1, 'App1', 1),
  (2, 'App2', 3),
  (3, 'App3', NULL),
  (4, 'App4', 5);

SELECT applicant FROM waitlist WHERE priority NOT IN (1, 2) AND priority IS
  NOT NULL
```

- A. App2
App3
App4
- B. App1
App2
App4
- C. (no rows)
- D. App2
App4

Block 6, Window Functions

11. What does this query return?

```
CREATE TABLE contest2 (id INTEGER, name TEXT, points INTEGER);
INSERT INTO contest2 VALUES
  (1, 'Fay', 70),
  (2, 'Gus', 70),
  (3, 'Hana', 65),
  (4, 'Ivo', 60);
```

```
SELECT name, points,
       RANK() OVER (ORDER BY points DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY points DESC) AS drnk
FROM contest2
ORDER BY points DESC, name
```

- A. Fay | 70 | 1 | 1
 Gus | 70 | 1 | 1
 Hana | 65 | 3 | 2
 Ivo | 60 | 4 | 3
- B. Fay | 70 | 1 | 1
 Gus | 70 | 1 | 1
 Hana | 65 | 2 | 3
 Ivo | 60 | 3 | 4
- C. Fay | 70 | 1 | 1
 Gus | 70 | 2 | 2
 Hana | 65 | 3 | 3
 Ivo | 60 | 4 | 4
- D. Fay | 70 | 1 | 1
 Gus | 70 | 1 | 1
 Hana | 65 | 2 | 2
 Ivo | 60 | 3 | 3

12. What does this query return?

```
CREATE TABLE hires (id INTEGER, team TEXT, name TEXT, hire_date TEXT);
INSERT INTO hires VALUES
  (1, 'ops', 'Dez', '2019-06-15'),
  (2, 'ops', 'Cole', '2021-03-01'),
  (3, 'ops', 'Ann', '2020-01-10');
```

```
SELECT team, name, hire_date,
       ROW_NUMBER() OVER (PARTITION BY team ORDER BY hire_date ASC) AS
       seniority
FROM hires
ORDER BY name
```

- A. ops | Ann | 2020-01-10 | 2
 ops | Cole | 2021-03-01 | 1
 ops | Dez | 2019-06-15 | 3
- B. ops | Ann | 2020-01-10 | 1
 ops | Cole | 2021-03-01 | 2
 ops | Dez | 2019-06-15 | 0
- C. ops | Ann | 2020-01-10 | 2
 ops | Cole | 2021-03-01 | 3

ops | Dez | 2019-06-15 | 1
D. ops | Ann | 2020-01-10 | 1
ops | Cole | 2021-03-01 | 2
ops | Dez | 2019-06-15 | 3

13. What does this query return?

```
CREATE TABLE track_plays (id INTEGER, artist TEXT, track TEXT, play_count
    INTEGER);
INSERT INTO track_plays VALUES
    (1, 'nova', 'Echo', 5000),
    (2, 'nova', 'Static', 7200),
    (3, 'nova', 'Drift', 7200),
    (4, 'nova', 'Halo', 4100);

SELECT artist, track, play_count,
       DENSE_RANK() OVER (PARTITION BY artist ORDER BY play_count DESC) AS
       drnk
FROM track_plays
ORDER BY artist, drnk, track
```

- A. nova | Drift | 7200 | 1
nova | Static | 7200 | 1
nova | Echo | 5000 | 3
nova | Halo | 4100 | 4
- B. nova | Drift | 7200 | 1
nova | Static | 7200 | 1
nova | Echo | 5000 | 2
nova | Halo | 4100 | 3
- C. nova | Drift | 7200 | 1
nova | Static | 7200 | 2
nova | Echo | 5000 | 3
nova | Halo | 4100 | 4
- D. nova | Halo | 4100 | 1
nova | Echo | 5000 | 2
nova | Drift | 7200 | 3
nova | Static | 7200 | 3

14. What does this query return?

```
CREATE TABLE grade_results (id INTEGER, school TEXT, grade_level INTEGER,
    student TEXT, score INTEGER);
INSERT INTO grade_results VALUES
    (1, 'lakeview', 5, 'Nia', 88),
    (2, 'lakeview', 5, 'Omar', 95),
    (3, 'lakeview', 6, 'Pat', 79),
    (4, 'lakeview', 6, 'Qi', 84);

SELECT school, grade_level, student, score,
    RANK() OVER (PARTITION BY school, grade_level ORDER BY score DESC) AS
    rnk
FROM grade_results
ORDER BY school, grade_level, rnk
```

- A. lakeview | 5 | Omar | 95 | 0
lakeview | 5 | Nia | 88 | 1
lakeview | 6 | Qi | 84 | 0
lakeview | 6 | Pat | 79 | 1
- B. lakeview | 5 | Omar | 95 | 2
lakeview | 5 | Nia | 88 | 1
lakeview | 6 | Qi | 84 | 2
lakeview | 6 | Pat | 79 | 1
- C. lakeview | 5 | Omar | 95 | 1
lakeview | 5 | Nia | 88 | 2
lakeview | 6 | Qi | 84 | 3
lakeview | 6 | Pat | 79 | 4
- D. lakeview | 5 | Omar | 95 | 1
lakeview | 5 | Nia | 88 | 2
lakeview | 6 | Qi | 84 | 1
lakeview | 6 | Pat | 79 | 2

Block 7, Strings, Numbers and Type Conversion

15. What does this query return?

```
CREATE TABLE queries (q TEXT);
INSERT INTO queries VALUES ('100% sure'), ('not sure'), ('50% maybe');

SELECT q FROM queries WHERE q LIKE '1__\% sure' ESCAPE '\'
```

- A. 100% sure
50% maybe
- B. not sure
- C. 100% sure
- D. (no rows)

16. What does this query return?

```
CREATE TABLE temps (city TEXT, change REAL);
INSERT INTO temps VALUES ('Oslo', -6.0), ('Cairo', 2.0), ('Quito', -0.5);
```

```
SELECT city FROM temps WHERE ABS(change) > 1 ORDER BY city
```

- A. Cairo
Oslo
Quito
- B. Oslo
- C. Cairo
Oslo
- D. Quito

Block 8, Data Modification and Constraints

17. A warehouse restocks several bins in one pass. The statements above run, then this query. What comes back?

```
CREATE TABLE parts (sku TEXT PRIMARY KEY, bin TEXT, qty INTEGER);
INSERT INTO parts VALUES ('A1', 'R1', 5), ('A2', 'R1', 8);
INSERT INTO parts VALUES ('A3', 'R2', 3), ('A4', 'R2', 6), ('A5', 'R3', 9);
```

```
SELECT * FROM parts ORDER BY sku
```

- A. A1 | R1 | 5
A2 | R1 | 8
A3 | R2 | 3
- B. A5 | R3 | 9
A4 | R2 | 6
A3 | R2 | 3
A2 | R1 | 8
A1 | R1 | 5
- C. A1 | R1 | 5
A2 | R1 | 8
A3 | R2 | 3
A4 | R2 | 6
A5 | R3 | 9
- D. A1 | R1 | 5
A2 | R1 | 8
A3 | R2 | 3
A4 | R2 | 6

18. A restaurant adds new dishes in one statement. The statements above run, then this query. What comes back?

```
CREATE TABLE menu (id INTEGER PRIMARY KEY, dish TEXT, price INTEGER);
INSERT INTO menu VALUES (1, 'soup', 8), (2, 'salad', 7);
INSERT INTO menu VALUES (3, 'steak', 22), (4, 'pasta', 14), (5, 'cake', 9);
```

```
SELECT dish FROM menu WHERE price > 10 ORDER BY dish
```

- A. pasta
- B. pasta
steak
- C. cake
pasta
steak
- D. steak
pasta

19. A clerk lists a new title before a price has been set, with no default defined on that column. The statements above run, then this query. What comes back?

```
CREATE TABLE books (id INTEGER PRIMARY KEY, title TEXT, price INTEGER);
INSERT INTO books VALUES (1, 'Atlas', 20), (2, 'Comet', 15);
INSERT INTO books (id, title) VALUES (3, 'Delta');

SELECT * FROM books ORDER BY id
```

- A. 1 | Atlas | 20
2 | Comet | 15
3 | Delta | NULL
- B. 1 | Atlas | 20
2 | Comet | 15
3 | Delta | 0
- C. 1 | Atlas | 20
2 | Comet | 15
- D. 1 | Atlas | 20
2 | Comet | 15
3 | Delta | 20

20. An airline books a passenger on a flight before a seat is picked, with no default defined on that column. The statements above run, then this query. What comes back?

```
CREATE TABLE bookings (id INTEGER PRIMARY KEY, flight TEXT, seat TEXT);
INSERT INTO bookings VALUES (1, 'AF100', '12A'), (2, 'AF100', '12B');
INSERT INTO bookings (id, flight) VALUES (3, 'AF100');

SELECT * FROM bookings ORDER BY id
```

- A. 1 | AF100 | 12A
2 | AF100 | 12B
- B. 1 | AF100 | 12A
2 | AF100 | 12B
3 | AF100 | NULL
- C. 1 | AF100 | 12A
2 | AF100 | 12B
3 | AF100 | 12A
- D. 1 | AF100 | 12A
2 | AF100 | 12B
3 | AF100 |

Answers and Explanations

Organized in the same order as the questions. Each option is explained, the correct one and the three distractors.

SQL Answers

1. Correct answer, **B**

A. (no rows)

This treats NOT b = 3 as a check for a missing value, as if it meant b IS NULL. NOT in front of an equality reverses a comparison between actual values, it does not test for NULL.

B. (correct) 4

Correct. Only row 4 has a in 1, 2 and, at the same time, a b that is not 3.

C. 1

4

This regroupes the condition as a = 1 OR (a = 2 AND NOT b = 3), moving the parentheses away from the OR. The query as written groups a = 1 OR a = 2 first, and applies NOT b = 3 to that whole group with AND.

D. 1

2

This drops the NOT and keeps the plain equality b = 3, so it keeps rows that match b = 3 instead of rows that avoid it. NOT reverses the condition it applies to, here it asks for rows where b is not 3.

NOT applies to the single condition or parenthesized group written right after it, and it reverses that condition rather than testing for a missing value. Parentheses fix which conditions belong to the OR, moving them changes which rows satisfy the AND that follows.

2. Correct answer, **B**

A. bowl

mug

plate

This keeps every price at or below the upper bound and forgets to enforce the lower bound, letting bowl in even though its price is below 10. BETWEEN checks both limits together.

B. (correct) mug

plate

Correct. BETWEEN includes both boundary values, so mug at 10 and plate at 50 both qualify, while bowl at 9 and jug at 51 fall outside the range.

C. (no rows)

This treats BETWEEN as excluding its two boundary values, dropping mug and plate because their price sits exactly on a limit. BETWEEN is inclusive on both ends, a value equal to either bound still counts as a match.

D. jug

mug

plate

This keeps every price at or above the lower bound and forgets to cap it at the upper bound, letting jug in even though its price is above 50. BETWEEN checks both limits together.

BETWEEN low AND high is shorthand for a column being greater than or equal to low and, at the same time, less than or equal to high, both boundary values are included in the match. Dropping either half of that combined check, or excluding the boundary values themselves, changes which rows pass.

3. Correct answer, A

A. (correct) M1 | Green

M1 | Red

M1 | Yellow

M2 | Red

M2 | Yellow

M3 | Red

Correct. M1 has run the fewest hours so it fits under all three thresholds, M2 fits under Yellow and Red, and M3, the most used, only fits under Red.

B. M2 | Green

M3 | Green

M3 | Yellow

This flips the inequality direction, matching thresholds whose max_hours is at or below the machine's hours_run instead of at or above, the opposite of a maintenance lookup.

C. (no rows)

This assumes hours_run must exactly equal max_hours, but the condition is an inequality, and none of the machines have run for exactly a threshold value.

D. M1 | Green

M2 | Yellow

M3 | Red

This assumes each machine can only match the tightest threshold it fits into, but the ON condition is satisfied by every threshold whose max_hours is at least the machine's hours_run, not only the lowest one.

The less a machine has been used, the more maintenance thresholds it satisfies under a $\leq$ comparison, since a small hours_run clears more of the max_hours limits. This is the same shape of inequality join as matching a score against a set of grade cutoffs.

4. Correct answer, D

A. Gate1 | Morning

Gate2 | Evening

This pairs the tables row by row as if zipping two lists together, which only produces as many rows as the shorter table and drops Gate3 and most of the combinations entirely.

B. (no rows)

This assumes a CROSS JOIN needs some equality condition to link the tables, but a CROSS JOIN has no condition at all, every row of one side pairs with every row of the other regardless of any matching column.

C. Gate1 | Evening

Gate1 | Morning

Gate2 | Evening

Gate2 | Morning

Gate3 | Evening

This undercounts the result as if it only had three stations plus two shifts minus one, five rows, instead of the full product of three times two, six rows, and it drops Gate3 with Morning entirely.

D. (correct) Gate1 | Evening

Gate1 | Morning

Gate2 | Evening

Gate2 | Morning

Gate3 | Evening

Gate3 | Morning

Correct. CROSS JOIN pairs every station with every shift, three stations times two shifts gives six combinations in total.

A *CROSS JOIN* always produces rows equal to the product of the two row counts, three stations and two shifts multiply to six, never a sum and never a row by row pairing that stops at the shorter table.

5. Correct answer, B

A. tia | 1 | 1
sam | 3 | 1

The counts are right but the members are not alphabetically ordered, sam comes before tia.

B. (correct) sam | 3 | 1
tia | 1 | 1

Correct. Sam has three loans but only one with a non-NULL returned value, and tia has a single loan that is also returned.

C. sam | 2 | 1
tia | 1 | 1

This drops one of sam loans from COUNT(*), as if a row with a NULL somewhere in it were invisible to COUNT(*), but COUNT(*) counts rows regardless of any NULL inside them.

D. sam | 3 | 3
tia | 1 | 1

This assumes COUNT(returned) equals COUNT(*), but two of sam loans have a NULL returned value and get skipped by COUNT on that column.

COUNT() counts rows no matter what NULLs they contain, while COUNT (column) only counts rows where that specific column is not NULL. A row with a NULL in one column still fully counts toward COUNT(*)*.

6. Correct answer, A

A. (correct) fail | 3
pass | 2

Correct. Ann, dee and eli score under 60 and fall into fail, while bo and cy score 60 or above and fall into pass.

B. pass | 2
fail | 3

The counts are right but the order is not, ORDER BY result sorts alphabetically and fail comes before pass.

C. fail | 2
pass | 3

This swaps the two counts, crediting fail with only two students when three of them, ann, dee and eli, score under 60.

D. fail | 3
pass | 3

This counts dee among the passing students, but 58 is below the 60 threshold in the CASE expression, so dee falls into fail rather than pass.

GROUP BY can group by the same expression used in the SELECT list, addressed either by repeating the expression or by its output alias. Each row is first classified by the CASE expression, and only afterward are the classified rows counted per group.

7. Correct answer, C

A. pop | 800
jazz | 1200
rock | NULL

The totals are correct but the events are not sorted alphabetically, jazz comes before pop before rock.

B. jazz | 1200
pop | 800

rock | 0

This treats a group with only NULL revenue as summing to zero, but SUM returns NULL, not zero, when there is nothing non-NULL to add.

C. (correct) jazz | 1200

pop | 800

rock | NULL

Correct. Jazz totals 1200, pop totals 800, and rock, whose two rows are both NULL, returns NULL.

D. jazz | NULL

pop | 800

rock | NULL

This treats the NULL row in jazz as poisoning its whole SUM, but SUM skips that row and keeps the 1200 from the other one.

A NULL value contributes nothing to a SUM, it is simply skipped, and a group made entirely of NULLs has nothing left after skipping, which is why SUM reports NULL rather than zero in that case.

8. Correct answer, D

A. A1 | 2

A2 | 1

This lists A2 as well, but A2 has only one withdrawal of 50 or more, which fails HAVING COUNT(*) >= 2, only A1 clears that bar.

B. A2 | 1

This keeps A2, whose single qualifying withdrawal does not meet the HAVING threshold, instead of A1, which has two.

C. A1 | 3

A2 | 1

This counts the 20 withdrawal for A1 too, but WHERE amount >= 50 removes it before grouping, leaving only two qualifying withdrawals for A1.

D. (correct) A1 | 2

Correct. WHERE keeps two A1 withdrawals and one A2 withdrawal, and HAVING then drops A2 for not reaching two.

WHERE removes the small withdrawals before grouping starts, and HAVING then removes any account whose remaining count still falls short of the threshold, the two conditions are checked at different stages.

9. Correct answer, D

A. P1

P2

P3

This assumes the NULL in the list is simply ignored, leaving NOT IN to behave as if the list only had A and B, but SQL does not drop NULL members from a list before evaluating it.

B. P3

This picks out a single row as if only D somehow escaped the NULL trap, but the NULL in the list affects every row identically, there is no partial escape.

C. P2

P3

This is the answer a straightforward reading of NOT IN would suggest, categories C and D are not A or B, but a NULL sitting inside the list turns every row unknown regardless of its own category.

D. (correct) (no rows)

Correct. Because the list itself contains a NULL, NOT IN can never be confirmed true for any row, so the query returns nothing even though P2 and P3 categories look like an obvious match at first glance.

NOT IN (A, B, NULL) unfolds into value <> A AND value <> B AND value <> NULL, and the last comparison is always unknown. Unknown combined with AND drags the whole expression to unknown or false depending on the other terms, but never to true, so a NULL anywhere inside a NOT IN list quietly empties the result for every row.

10. Correct answer, D

A. App2
App3
App4

This keeps App3 despite the explicit IS NOT NULL guard, forgetting that a missing priority fails that check regardless of what NOT IN alone would decide.

B. App1
App2
App4

This keeps App1 as if priority 1 slipped past NOT IN (1, 2), but 1 is literally one of the excluded values and fails that test directly.

C. (no rows)

This assumes the NULL priority anywhere in the table defeats the AND for every row, but the IS NOT NULL guard exists precisely to handle that row without affecting the others.

D. (correct) App2
App4

Correct. App2 and App4 have a priority outside the excluded list, App1 matches the excluded value 1, and App3 fails the explicit IS NOT NULL guard.

Adding an explicit priority IS NOT NULL guard alongside NOT IN protects against the usual trap, here the list itself has no NULL member so NOT IN alone would already work for the defined priorities, and the guard simply confirms that App3 is dropped because its priority is missing, not because of anything inside the list.

11. Correct answer, A

A. (correct) Fay | 70 | 1 | 1
Gus | 70 | 1 | 1
Hana | 65 | 3 | 2
Ivo | 60 | 4 | 3

Correct. Both functions tie Fay and Gus at 1, then RANK jumps to 3 for Hana because two rows sit above it, while DENSE_RANK moves to 2 with no gap.

B. Fay | 70 | 1 | 1
Gus | 70 | 1 | 1
Hana | 65 | 2 | 3
Ivo | 60 | 3 | 4

This swaps the two columns for the non tied rows, giving RANK the gapless numbers that belong to DENSE_RANK and handing DENSE_RANK the skipped numbers that belong to RANK.

C. Fay | 70 | 1 | 1
Gus | 70 | 2 | 2
Hana | 65 | 3 | 3
Ivo | 60 | 4 | 4

This applies ROW_NUMBER logic to both columns, breaking the tie between Fay and Gus instead of letting RANK and DENSE_RANK agree on 1 for both of them.

D. Fay | 70 | 1 | 1
Gus | 70 | 1 | 1
Hana | 65 | 2 | 2
Ivo | 60 | 3 | 3

This applies DENSE_RANK logic to both columns, so RANK never skips ahead after the tie even though it counts rows above the current one rather than distinct values.

RANK and DENSE_RANK only disagree once a tie has happened, and only about what comes right after it. RANK counts rows above the current one, so a tie of size two is followed by a jump of two, while DENSE_RANK counts distinct values and always moves to the next integer.

12. Correct answer, C

A. ops | Ann | 2020-01-10 | 2
ops | Cole | 2021-03-01 | 1
ops | Dez | 2019-06-15 | 3

This ranks the most recent hire first, the opposite of ASC, so Cole would get seniority 1 instead of the last position that a newer hire date actually earns.

B. ops | Ann | 2020-01-10 | 1
ops | Cole | 2021-03-01 | 2
ops | Dez | 2019-06-15 | 0

This starts the count at 0 for the earliest hire instead of 1, treating ROW_NUMBER as a zero based index rather than the one based row position SQL defines.

C. (correct) ops | Ann | 2020-01-10 | 2
ops | Cole | 2021-03-01 | 3
ops | Dez | 2019-06-15 | 1

Correct. The window orders by hire_date, so Dez, the earliest hire, gets seniority 1, while the final ORDER BY name displays the rows alphabetically.

D. ops | Ann | 2020-01-10 | 1
ops | Cole | 2021-03-01 | 2
ops | Dez | 2019-06-15 | 3

This assumes seniority follows the alphabetical order the rows are displayed in, as if the window ORDER BY copied the outer ORDER BY name instead of using hire_date.

The ORDER BY inside OVER and the ORDER BY at the end of the query are unrelated, one decides the values a window function computes and the other decides the order rows are displayed in. Sorting the display alphabetically does not change which hire counts as the most senior.

13. Correct answer, B

A. nova | Drift | 7200 | 1
nova | Static | 7200 | 1
nova | Echo | 5000 | 3
nova | Halo | 4100 | 4

This is what RANK would produce, also tying the top two tracks at 1 but then skipping ahead to 3 for the next track instead of the gapless 2 DENSE_RANK gives.

B. (correct) nova | Drift | 7200 | 1
nova | Static | 7200 | 1
nova | Echo | 5000 | 2
nova | Halo | 4100 | 3

Correct. Drift and Static tie for the most plays and both get 1, then DENSE_RANK moves straight to 2 for Echo with no gap, and 3 for Halo.

C. nova | Drift | 7200 | 1
nova | Static | 7200 | 2
nova | Echo | 5000 | 3
nova | Halo | 4100 | 4

This is what ROW_NUMBER would produce, splitting the tied top tracks into two different numbers instead of letting them share the same rank.

D. nova | Halo | 4100 | 1
nova | Echo | 5000 | 2
nova | Drift | 7200 | 3
nova | Static | 7200 | 3

This orders play_count ascending instead of descending, so the least played track ranks first, the opposite of what DESC in the window ORDER BY specifies.

DENSE_RANK counts distinct play counts, not rows, which is why a tie for the top spot is immediately followed by the very next integer rather than a number that skips ahead to account for both tied rows.

14. Correct answer, D

A. lakeview | 5 | Omar | 95 | 0
lakeview | 5 | Nia | 88 | 1
lakeview | 6 | Qi | 84 | 0
lakeview | 6 | Pat | 79 | 1

This starts each partition at 0 instead of 1, treating RANK as a zero based index rather than the one based position SQL actually assigns.

B. lakeview | 5 | Omar | 95 | 2
lakeview | 5 | Nia | 88 | 1
lakeview | 6 | Qi | 84 | 2
lakeview | 6 | Pat | 79 | 1

This orders score ascending instead of descending inside each grade, so the lower score ranks first rather than the higher one.

C. lakeview | 5 | Omar | 95 | 1
lakeview | 5 | Nia | 88 | 2
lakeview | 6 | Qi | 84 | 3
lakeview | 6 | Pat | 79 | 4

This partitions by school alone, letting grade 5 and grade 6 share one continuous ranking instead of each grade level restarting its own count.

D. (correct) lakeview | 5 | Omar | 95 | 1
lakeview | 5 | Nia | 88 | 2
lakeview | 6 | Qi | 84 | 1
lakeview | 6 | Pat | 79 | 2

Correct. Grade 5 and grade 6 at lakeview form separate partitions even though they share a school, so each grade ranks its own two students starting at 1.

A two column PARTITION BY forms a group for every distinct pair of values, so rows from grade 5 never compete with rows from grade 6 even under the same school, they are scored as if they belonged to entirely separate tables.

15. Correct answer, C

A. 100% sure
50% maybe

This includes '50% maybe' as well, but it starts with 5 rather than 1, and it ends in 'maybe' rather than 'sure', so it fails the pattern on both counts.

B. not sure

This picks 'not sure' instead, but that string does not start with the digit 1 followed by two more characters and a percent sign at all.

C. (correct) 100% sure

Correct. The two underscores each match one character, 0 and 0, the escaped percent matches the literal percent sign, and the trailing text matches exactly, so only '100% sure' fits the pattern.

D. (no rows)

This assumes no row matches, but '100% sure' fits the pattern exactly, one 1, two wildcard characters covering the remaining zeros, then a literal percent sign and the word sure.

Underscore in LIKE matches exactly one character each, so two consecutive underscores require exactly two characters there, while an escaped percent sign in the same pattern still means a literal percent, not a wildcard.

16. Correct answer, C

A. Cairo

Oslo

Quito

This includes Quito, but its change is only 0.5 in absolute value, which does not exceed 1.

B. Oslo

This keeps only Oslo, but Cairo's change of 2.0 also has an absolute value above 1 and belongs in the result too.

C. (correct) Cairo

Oslo

Correct. ABS(change) is 6.0 for Oslo and 2.0 for Cairo, both above 1, while Quito's 0.5 is not, so Oslo and Cairo qualify, sorted alphabetically.

D. Quito

This keeps only Quito, but its small change is exactly the one that fails the threshold, while the two larger changes pass it.

ABS lets a single comparison catch large changes in either direction, a threshold on ABS(column) treats a big negative change the same as a big positive one.

17. Correct answer, C

A. A1 | R1 | 5

A2 | R1 | 8

A3 | R2 | 3

This keeps only the first row of the second, three row INSERT statement, as if a multi row INSERT only added the first pair of values listed.

B. A5 | R3 | 9

A4 | R2 | 6

A3 | R2 | 3

A2 | R1 | 8

A1 | R1 | 5

This has all five rows but sorted from Z to A instead of the ascending order ORDER BY sku gives by default.

C. (correct) A1 | R1 | 5

A2 | R1 | 8

A3 | R2 | 3

A4 | R2 | 6

A5 | R3 | 9

Correct. A single INSERT statement can list several rows separated by commas, and all three rows in the second statement are added along with the two from the first, for five parts sorted by sku.

D. A1 | R1 | 5
A2 | R1 | 8
A3 | R2 | 3
A4 | R2 | 6

This keeps only the first two rows of the second INSERT, dropping A5 as though the statement had stopped short.

INSERT INTO table VALUES (...), (...), (...) adds every row listed, not just the first one, in one statement. The comma separates complete rows, each in parentheses, and every one of them is inserted before the statement finishes.

18. Correct answer, B

A. pasta

This drops steak, as if only the first of the three rows added by the second INSERT had been kept.

B. (correct) pasta
steak

Correct. Five menu items exist once both INSERT statements run, and only steak at 22 and pasta at 14 clear the price greater than 10 filter, listed alphabetically.

C. cake
pasta
steak

This includes cake, which costs 9 and does not pass the price greater than 10 filter.

D. steak
pasta

This has the right two dishes but in the wrong order, listing steak before pasta instead of alphabetically.

A WHERE clause is applied after the table already holds every row from every INSERT in the schema, it filters the finished data rather than the statements that built it. Rows from a multi row INSERT are each tested against the condition individually, only cake among the three new rows fails it here.

19. Correct answer, A

A. (correct) 1 | Atlas | 20
2 | Comet | 15
3 | Delta | NULL

Correct. price has no DEFAULT clause, so leaving it out of the column list for Delta stores NULL there, while the title and id are set as given.

B. 1 | Atlas | 20
2 | Comet | 15
3 | Delta | 0

This assumes an omitted numeric column becomes 0, but with no DEFAULT clause on price, the column is left as NULL instead.

C. 1 | Atlas | 20
2 | Comet | 15

This drops the Delta row entirely, as if leaving price out of the column list made the insert fail rather than simply leave that column empty.

D. 1 | Atlas | 20
2 | Comet | 15
3 | Delta | 20

This copies the price from the row above it, when nothing in the statement supplies a value for Delta price.

A column left out of the INSERT column list becomes NULL unless the table defines a DEFAULT for it, there is no other fallback. price here has no default, so Delta simply has no price recorded, rather than a zero or a copied value.

20. Correct answer, B

A. 1 | AF100 | 12A

2 | AF100 | 12B

This drops booking 3 entirely, as if leaving seat out of the column list made the insert fail rather than leave that column empty.

B. (correct) 1 | AF100 | 12A

2 | AF100 | 12B

3 | AF100 | NULL

Correct. seat has no DEFAULT clause, so leaving it out of the column list stores NULL for booking 3, while id and flight are set as given.

C. 1 | AF100 | 12A

2 | AF100 | 12B

3 | AF100 | 12A

This copies a seat from an earlier booking, when nothing in the statement supplies a seat value for the third row.

D. 1 | AF100 | 12A

2 | AF100 | 12B

3 | AF100 |

This treats the missing seat as an empty string rather than NULL, but an omitted value is stored as NULL, not as text with no characters.

A column absent from the INSERT column list is set to NULL when the table gives it no DEFAULT, regardless of what values sit in other rows of that column. Nothing about the values already stored for other bookings carries over to a row that never mentions the column.

That was 20 of the 502 SQL questions.

The full bank is one PDF laid out exactly like this sample, all 502 questions first and the complete answer key after, with a reason for each of the four options. Every snippet in it was run the same way before its answer was written.

Get it for 9 euros at <https://ko-fi.com/s/5128d84869>, or all three languages in one PDF for 19 euros at <https://ko-fi.com/s/fa83c4ed59>.

The same 20 questions can be played in a browser at <https://thibaudlepan77-svg.github.io/interview-questions-verified/>. If an answer looks wrong to you, open an issue on the repository linked from that page with the question number and what your own run printed.